

The QueryGraph Stack

A governed semantic lakehouse for agentic AI

Grust · TypeSec · LakeCat · Sail · QueryGraph

Stack review · QueryGraph 0.3.0 “Goshawk” · querygraph.ai

The thesis: prove what the agent did

- Enterprise AI's differentiator is not a bigger context window — it is a **verifiable chain** from question to answer.
- Every answer carries: the semantics used, the policies that allowed it, the sources touched, **and the sources denied** — with receipts.
- “The answer used OSI metric X, resolved to Sail table Y, under capability C and odr1:read, emitting OpenLineage run R anchored by attestation A — and source Z was denied, with a receipt.”
- No mainstream agent framework offers that sentence.

Five components, one evidence chain

Component	Role	Release
Grust	Backend-neutral property graph + GQL/Cypher reads	0.11.0 “Crab”
TypeSec	Capabilities as types; TypeDID signed envelopes	0.11.0 “Burano”
LakeCat	Rust Iceberg REST catalog; QueryGraph bootstrap bundles	0.2.1 “Lynx”
Sail (fork)	Spark-compatible engine + Cypher extension	branch grust
QueryGraph	The governed semantic layer, Rust + Python	0.3.0 “Goshawk”

Grust: the graph substrate

- Graph = nodes + edges; ids, labels, typed properties — backend-neutral.
- One facade crate, many stores: memory, SurrealDB, FalkorDB, HelixDB, LanceDB, PostgreSQL (incl. SQL/PGQ), Turso, **Sail DataFrames**.
- grust-cypher: Cypher/GQL reads over any store.
- QueryGraph's semantic graph — datasets, fields, ontology terms, agents, policies — is a Grust graph.
- In flight: the Full39075 GQL profile (CALL subqueries, TVFs, shortestPath, passthrough) + atomic batch transactions.

TypeSec: security in the type system

- Capability<CanWrite, Report> is an **unforgeable proof** — crate-private constructor, phantom types, sealed permissions.
- No capability $\Rightarrow$ the guarded function **does not exist** for your code. Violations are compile errors.
- TypeDID: Ed25519 keys from seeds, did:key documents, signed + encrypted agent envelopes with audit-safe attestations.
- In flight: framework interop plane (OpenAI/Anthropic/LangChain/Pydantic-AI guards), deny-by-default mcp-gate, enforcement proxy, signed receipts.

LakeCat: the catalog boundary

- Rust-native Apache Iceberg REST catalog (/catalog/v1) — standard clients just work.
- Catalog state, Sail planning, TypeSec receipts, and Grust projection bind to the **same accepted table transition**.
- /querygraph/v1/bootstrap: live tables projected into Croissant, CDIF, OSI, ODRL, OpenLineage, and a Grust-ready envelope — one shared wire crate, no copied formats.
- In flight: Iceberg REST hardening, fail-closed v4 validation, recorded release-candidate proof.

Sail + the Cypher extension

- Spark-compatible lakehouse engine: typed Parquet tables, Spark Connect, PySpark — the compute substrate.
- The fork compiles a **Cypher graph-query surface into Sail itself** (~5,600 lines: parser AST, analyzer, plan resolver).
- Reuses Grust's property-graph model — the same semantic graph is MATCH-able from any Spark session.
- QueryGraph's audit trail (`qg_audit`) lives next to the data it audits.

QueryGraph: four projections + business semantics

- Every dataset described four ways: **Semantic Croissant** (ML metadata), **CDIF** (discovery/access), **W3C DID** (deterministic identity), **ODRL** (rights).
- **OSI** models map business terms → governed Sail columns, with per-dialect SQL and ontology terms.
- Governance is a dual gate: RBAC **and** ODRL must both allow. Denials are receipts, not errors.
- Lineage: OpenLineage events (official-schema valid, UUIDv5 run ids) + Ed25519-anchored attestations.

Goshawk (0.3.0): the interoperability release

- **Real Ed25519 across languages:** Python signs → Rust verifies, Rust signs → Python verifies; same seed ⇒ same did:key. Tampering fails.
- **HTTP /v1 API:** bundles, story, model registry + search, answer, envelope audit — with TypeDID envelope auth (path- and body-bound).
- **MCP servers in both languages** — Claude, LangChain, PydanticAI, LlamaIndex, CrewAI reach everything with zero adapters.
- **A2A agent card** at `/.well-known/agent-card.json` — identical skills from both implementations, asserted by tests.

The governed navigator loop (0.4-dev)

question → semantic search (synonyms, bigrams) → **RBAC+ODRL receipts** → SQL plans over allowed sources → synthesis → **signed envelope + OpenLineage + attestation**

- Denied sources are named in the prompt as off-limits — never planned.
- LLM is any `Callable[[str], str]`: Ollama, vLLM, llama.cpp, LM Studio via one OpenAI-compatible helper; `llm=None` is the deterministic baseline.
- Same governance with or without a model in the loop.

Held equivalent by tests, not discipline

- Navigator bundles: byte-identical modulo timestamps.
- QGLake story: same roster, only the restricted broker denied, field-identical attestation schemas.
- Crypto: both directions verified live; fixtures pin shared did:key and UUIDv5 derivations.
- OpenLineage events from **both** CLIs validate against the official schema.
- Live auth: Python-minted header → 200; without → 401 + receipt.
- 49 Python + 40 Rust tests; CI on every push.

Operating the stack

```
cargo run -- serve --require-auth      # /v1 + agent card
cargo run -- mcp-serve                  # MCP over stdio (Rust)
uv run querygraph mcp-serve --osi m.yaml
uv run querygraph answer --question "..."
```

- Sibling checkouts: ~/src/{querygraph,grust,lakecat}; TypeSec from crates.io.
- Coordinated codenamed releases; CHANGELOGs; versioned book artifacts; GitHub Actions CI in every repo.

Roadmap

- Navigator loop: live LLM runs under identical receipts; Rust parity.
- Remaining /v1: lineage queries, audit verification, access explanation.
- Adopt TypeSec-next: interop plane, mcp-gate, enforcement proxy.
- Polaris SemanticModel + /navigator-bundle (LakeCat first); OSIMetricFacet → OpenLineage; dbt/Cube importers; ADBC path.
- The benchmark: **how much does a governed semantic layer improve agent accuracy over the same lakehouse?**

Links

- Meta-repo: github.com/querygraph/querygraph
- Implementations: [querygraph/qg-rust](#) · [querygraph/qg-python](#) (releases v0.3.0 “Goshawk”)
- Substrate: [querygraph/grust](#) · [querygraph/typesec](#) · [querygraph/lakecat](#) · [lakehq/sail](#)
- Books: the dedicated QueryGraph book ([qg-rust/docs/book](#)) and **The QueryGraph Stack** guide ([qg-rust/docs/guide](#))
- Standards: Croissant · CDIF · DID · ODRL · OSI · OpenLineage · MCP · A2A

querygraph.ai